LKPD 8 - Docker Swarm 3 Node (AWS Academy)

Oct 12, 2026 · @nugteam1

Identitas LKPD

Komponen	Keterangan
Mata Pelajaran	DevOps Engineer
Topik	Docker Swarm Multi Node (1 manager + 2 worker) di AWS
Kelas / Semester	_____
Nama Siswa	_____
Kelompok	_____
Alokasi Waktu	5 x 45 menit
Lingkungan Praktik	AWS Academy Learner Lab, 3 instance EC2 Ubuntu, IAM role LabInstanceProfile

Tujuan Pembelajaran

Setelah menyelesaikan LKPD ini, siswa mampu:

- Menyiapkan 3 instance EC2 Ubuntu untuk cluster Docker Swarm dan 1 instance terpisah untuk database, beserta security group-nya.
- Membentuk cluster swarm: 1 manager dan 2 worker.
- Men-deploy service dengan 3 replika yang tersebar di beberapa node, lalu membuktikan load balancing routing mesh.
- Melakukan scale, rolling update, dan drain node.
- Menempatkan MariaDB di instance tersendiri, menghubungkan replika web ke database itu, dan men-deploy web dengan `docker stack deploy`.

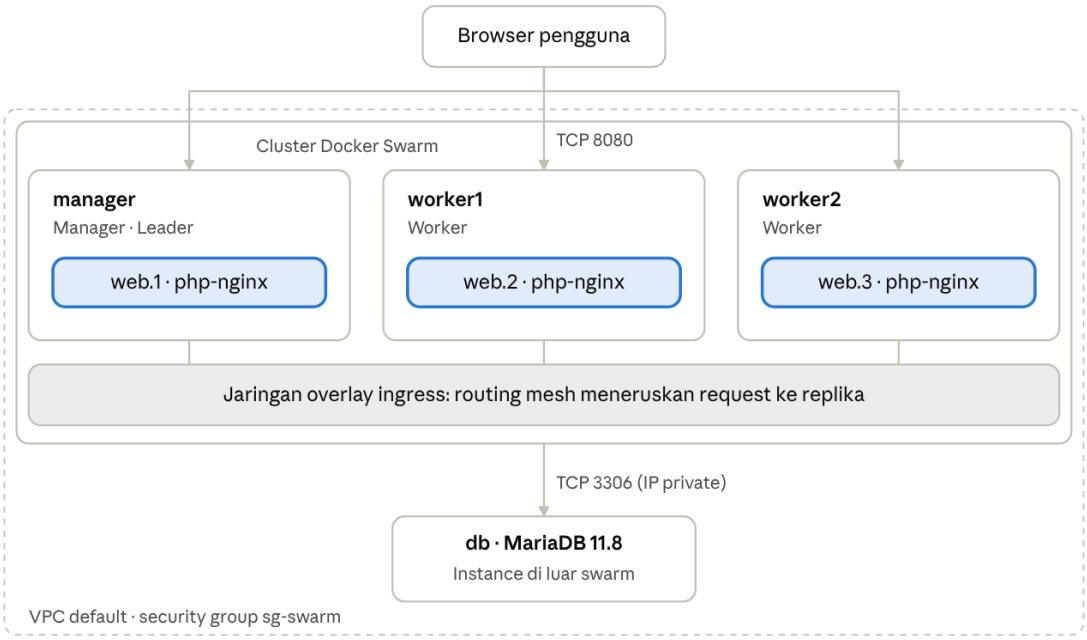
Skenario

Tiga instance EC2 Ubuntu di AWS Academy dijadikan satu cluster Docker Swarm: 1 manager dan 2 worker. Berbeda dari praktik single node, replika kini berjalan di mesin yang berlainan. Database MariaDB berjalan di instance keempat yang berdiri sendiri di luar swarm.

- `docker swarm init --advertise-addr <IP-PRIVATE>` di manager membuat cluster. Kedua worker bergabung memakai token lewat port 2377 milik manager. Komunikasi antar node memakai IP private di VPC yang sama.

- `docker service create` dan `docker service scale` menjalankan 3 container `trafex/php-nginx` (web.1, web.2, web.3). Swarm menyebarkannya ke node yang tersedia, dan tiap container berisi nginx dan PHP-FPM.
- Port 8080 dipegang oleh routing mesh di setiap node. Request ke IP node mana pun diteruskan ke salah satu replika secara bergantian (round-robin) melalui jaringan overlay `ingress`.
- Instance memakai IAM role `LabInstanceProfile` sesuai ketentuan Learner Lab. Swarm sendiri tidak memerlukannya; yang menentukan koneksi antar node adalah security group.

Port 8080 di node mana pun menjangkau semua replika web



arsitektur cluster · 3 node, 3 replika web

Kotak berwarna adalah replika service `web`, kotak polos adalah instance database di luar swarm, dihubungi lewat IP private pada port 3306. Sebaran replika pada gambar hanya contoh, karena swarm yang menentukan penempatannya.

Persiapan Infrastruktur di AWS Academy

A. Start Learner Lab

1. Login ke AWS Academy, buka course, pilih **Learner Lab**.
2. Klik **Start Lab**. Tunggu indikator AWS berwarna hijau, lalu klik **AWS** untuk membuka console.
3. Pastikan region yang dipakai sama untuk semua resource (umumnya `us-east-1`).

B. Buat Security Group

EC2 → Security Groups → Create security group. Nama: `sg-swarm`, VPC: default.

Tipe	Protokol	Port	Sumber	Fungsi
SSH	TCP	22	My IP (atau 0.0.0.0/0 bila IP berubah-ubah)	Akses remote
Custom TCP	TCP	2377	`sg-swarm` (security group itu sendiri)	Komunikasi manajemen cluster
Custom TCP	TCP	7946	`sg-swarm`	Discovery antar node
Custom UDP	UDP	7946	`sg-swarm`	Discovery antar node
Custom UDP	UDP	4789	`sg-swarm`	Overlay network (VXLAN)
Custom TCP	TCP	8080	0.0.0.0/0	Akses web dari browser
Custom TCP	TCP	3306	sg-swarm	Akses database dari node swarm

Catatan: rule 2377, 7946, dan 4789 sumbernya diisi dengan security group itu sendiri, bukan 0.0.0.0/0, supaya hanya antar node yang boleh berkomunikasi. Buat dulu security group-nya, lalu edit inbound rules setelah tersimpan agar bisa memilih dirinya sendiri sebagai sumber. Rule 3306 juga bersumber security group itu sendiri, sehingga database hanya dapat diakses dari instance yang memakai sg-swarm.

C. Launch 4 Instance EC2

EC2 → Launch instances. Isi:

- **Name**: `swarm-node`. Nama diganti per instance sesudah launch (`manager`, `worker1`, `worker2`, db).
- **AMI**: Ubuntu Server 24.04 LTS.
- **Instance type**: `t3.small` (2 GB RAM, cukup untuk MariaDB dan PHP). `t2.micro` boleh untuk latihan ringan.
- **Key pair**: `vockey`.
- **Network settings**: VPC default, subnet yang sama, pilih security group `sg-swarm`, auto-assign public IP aktif.
- **Advanced details → IAM instance profile**: pilih `LabInstanceProfile`.
- **Number of instances**: 4 (manager, worker1, worker2, db).

Setelah berjalan, ubah kolom Name menjadi `manager`, `worker1`, `worker2`, serta db, lalu catat IP-nya.

Node	Peran	IP Private	IP Public
manager	Manager	_____	_____
worker1	Worker	_____	_____
worker2	Worker	_____	_____
db	Database (di luar swarm)	_____	_____

D. Akses SSH

Download `labsuser.pem` dari menu **AWS Details** di halaman Learner Lab, lalu dari terminal:

```
chmod 400 labsuser.pem
ssh -i labsuser.pem ubuntu@<IP-PUBLIC-NODE>
```

Buka 4 terminal, satu per instance. Alternatif tanpa key: EC2 → pilih instance → **Connect** → **EC2 Instance Connect**.

IP public berubah setiap lab di-stop lalu di-start lagi, sedangkan IP private tetap. Karena itu seluruh konfigurasi swarm memakai **IP private**.

Langkah Kerja

Tanda **[semua node]** berarti perintah dijalankan di manager, worker1, dan worker2. Instance db tidak ikut swarm, sehingga hanya dipakai pada Langkah 1 dan Tugas. Tanda **[manager]** berarti hanya di node manager. Ganti teks dalam `<...>` dengan IP milik Anda.

Langkah 1. Atur hostname dan instal Docker [semua instance, termasuk db]

Hostname diatur lebih dulu supaya nama node di swarm mudah dibaca. Ganti `manager` dengan `worker1` / `worker2` atau db sesuai instance.

```
sudo hostnamectl set-hostname manager
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker ubuntu
```

Keluar dari SSH lalu masuk lagi agar grup `docker` aktif, kemudian cek:

```
docker --version
docker run --rm hello-world
```

Opsional, percepat deploy dengan menarik image di semua node:

```
docker pull trafex/php-nginx
```

Langkah 2. Inisialisasi swarm [manager]

Gunakan IP private manager. Flag `--advertise-addr` menentukan alamat yang dipakai node lain untuk menghubungi manager.

```
docker swarm init --advertise-addr <IP-PRIVATE-MANAGER>
```

Salin perintah `docker swarm join --token ...` dari output. Jika hilang, tampilkan lagi dengan:

```
docker swarm join-token worker
```

Langkah 3. Gabungkan worker [worker1 dan worker2]

```
docker swarm join --token <TOKEN> <IP-PRIVATE-MANAGER>:2377
```
Jika muncul pesan `This node joined a swarm as a worker`, berhasil. Jika timeout, periksa kembali inbound rule port 2377 pada `sg-swarm`.

Kembali ke manager, cek anggota cluster:

```
docker node ls
```
Pastikan ada 3 node berstatus `Ready` / `Active`, dan hanya manager yang bertanda `Leader`. Perintah `docker node ls` tidak bisa dijalankan dari worker.

Langkah 4. Siapkan konten web [semua node]

Pada bind mount, folder sumber harus ada di setiap node yang menjalankan replika. Karena itu folder dan file berikut dibuat identik di ketiga node.

```
mkdir -p /home/ubuntu/mywww
cat > /home/ubuntu/mywww/index.php <<'EOF'
<?php
$hostname = gethostname();
$ip = gethostbyname($hostname);
?>
<!DOCTYPE html>
<html>
<head><title>Info Server</title></head>
<body style="font-family: sans-serif;">
  <h2>Info Server</h2>
  <p>Hostname: <b><?= htmlspecialchars($hostname) ?></b></p>
  <p>IP server: <b><?= htmlspecialchars($ip) ?></b></p>
  <p>IP client: <b><?= htmlspecialchars($_SERVER['REMOTE_ADDR']) ?></b></p>
</body>
</html>
EOF
```

Langkah 5. Buat service [manager]

```
docker service create --name web -p 8080:8080 \
  --mount type=bind,source=/home/ubuntu/mywww,target=/var/www/html \
  trafex/php-nginx
```
Cek dengan

`docker service ls`

dan

`docker service ps web`.

Saat ini baru ada 1 replika.

Langkah 6. Scale menjadi 3 replika [manager]

```
docker service scale web=3
docker service ps web
```

Perhatikan kolom **NODE**. Swarm menyebar replika ke node yang berbeda. Isi tabel pengamatan:

Replika	Berjalan di node	Container ID (hostname)
web.1	_____	_____
web.2	_____	_____
web.3	_____	_____

Container ID dapat dilihat dengan `docker ps --format 'table {{.ID}}\t{{.Names}}'` pada masing-masing node.

Langkah 7. Uji routing mesh dan load balancing

Buka di browser (port 8080 sudah dibuka di security group):

- `http://<IP-PUBLIC-MANAGER>:8080`
- `http://<IP-PUBLIC-WORKER1>:8080`
- `http://<IP-PUBLIC-WORKER2>:8080`

Semua alamat menampilkan halaman yang sama karena routing mesh membuka port 8080 di setiap node, walaupun node itu tidak menjalankan replika. Request lalu diteruskan ke salah satu replika secara bergantian (round-robin). Refresh berulang kali (atau buka beberapa tab / jendela incognito) sampai hostname berubah.

Uji lewat `curl` dari manager:

```
for i in {1..6}; do curl -s http://<IP-PRIVATE-WORKER1>:8080/index.php | grep -o
'<b>[^<]*</b>' | head -1; done
```

Hostname yang tampil adalah container ID, dan IP server adalah IP container di jaringan overlay `ingress`. IP client yang tampil bukan IP asli pengunjung, melainkan IP milik jaringan ingress karena request melewati routing mesh.

Langkah 8. Scale up dan scale down [manager]

```
docker service scale web=6
docker service ps web
docker service scale web=3
```

Langkah 9. Rolling update [manager]

Perintah ini memulai ulang replika satu per satu dengan jeda 10 detik. Selama proses berjalan, jalankan `curl` pada terminal lain dan amati apakah layanan tetap menjawab.

```
docker service update --force --update-parallelism 1 --update-delay 10s web
```

Langkah 10. Uji ketahanan cluster

a. Container mati. Di salah satu worker, hapus paksa container web:

```
docker ps
docker rm -f <CONTAINER-ID-WEB>
```
Di manager, jalankan `docker service ps web` beberapa kali. Swarm membuat replika pengganti secara otomatis.

b. Node di-drain. Drain berarti node tidak boleh menjalankan task, misalnya saat maintenance.

```
docker node update --availability drain worker2
docker service ps web
```
Replika yang ada di worker2 berpindah ke node lain. Aktifkan kembali:

```
docker node update --availability active worker2
```
Replika tidak otomatis kembali ke worker2. Untuk menyebar ulang: `docker service update --force web`.

Langkah 11. Pembersihan

Hapus service:

```
docker service rm web
```
Jangan keluar dari swarm dulu bila akan mengerjakan Tugas. Setelah semua selesai:

```
# di worker1 dan worker2
docker swarm leave
# di manager
docker swarm leave --force
```

Tugas

Deploy aplikasi appKantin (PHP + MySQL) di cluster: 3 replika web tersebar di 3 node, semuanya memakai **satu** database MariaDB 11.8 yang berjalan di **instance tersendiri** (instance `db`, di luar swarm). Menurut README-nya, aplikasi ini membuat database `kantin_db` beserta tabel dan data contoh secara otomatis saat pertama kali dibuka, sehingga tidak perlu import file SQL.

Pastikan swarm dari Langkah Kerja masih aktif dan service `web` sudah dihapus (`docker service rm web`).

Tugas 1. Jalankan MariaDB di instance db [db]

Docker sudah terpasang di instance ini pada Langkah 1. Data disimpan di volume Docker agar tidak hilang saat container dibuat ulang. Port 3306 dipublikasikan supaya bisa dijangkau node swarm; di security group, port itu hanya terbuka untuk `sg-swarm`.

```
docker run -d --name db --restart unless-stopped -p 3306:3306 \
  -e MARIADB_ROOT_PASSWORD=rahasia \
  -v dbdata:/var/lib/mysql \
  mariadb:11.8

docker ps
```
Catat IP private instance `db` pada tabel di bagian Persiapan.

Tugas 2. Uji koneksi dari node swarm [manager]

```
docker run --rm mariadb:11.8 mariadb -h <IP-PRIVATE-DB> -uroot -prahasia -e "SELECT VERSION();"
```

Jika versi MariaDB tampil, jalur jaringan dan password sudah benar. Jika timeout, periksa rule TCP 3306 pada `sg-swarm` dan pastikan instance `db` memakai security group yang sama.

Tugas 3. Ambil aplikasi dan atur koneksi database [manager, worker1, worker2]

Karena memakai bind mount, kode aplikasi harus ada di ketiga node swarm dengan isi yang sama.

```
git clone https://github.com/paknux/appKantin /home/ubuntu/appKantin
nano /home/ubuntu/appKantin/config/db.php
```

Ubah nilai berikut. `DB_HOST` diisi IP private instance `db`, bukan IP public. IP private tidak berubah saat instance di-stop lalu di-start lagi, sedangkan IP public berubah.

```
$DB_HOST = "<IP-PRIVATE-DB>";
$DB_USER = "root";
$DB_PASS = "rahasia";
$DB_NAME = "kantin_db";
```

Tugas 4. Jalankan aplikasi dengan 3 replika [manager]

```
docker service create --name kantin \
  --replicas 3 -p 8080:8080 \
  --mount type=bind,source=/home/ubuntu/appKantin,target=/var/www/html \
  trafex/php-nginx

docker service ps kantin
```

Buka `http://<IP-PUBLIC-MANAGER>:8080`. Pembukaan pertama membuat database dan tabel, sehingga bisa sedikit lebih lama.

Tugas 5. Verifikasi

1. Tambahkan satu menu baru lewat halaman aplikasi.
2. Refresh atau buka dari IP public worker1 dan worker2. Menu yang sama harus muncul di semua alamat, karena semua replika memakai database yang sama.
3. Di instance `db`, periksa isi database:

```
docker exec -it db mariadb -uroot -prahasia -e "USE kantin_db; SHOW TABLES; SELECT COUNT(*) FROM menu;"
```

4. Restart container database lalu pastikan data tidak hilang:

```
docker restart db
```

Tugas 6 (tantangan). Ubah web menjadi stack

Hapus service `kantin` (`docker service rm kantin`), lalu buat file `kantin-stack.yml` di manager. Database tidak ikut stack karena berada di instance sendiri.

```
services:
  web:
    image: trafex/php-nginx
    ports:
      - "8080:8080"
```

```
    volumes:
      - /home/ubuntu/appKantin:/var/www/html
    deploy:
      replicas: 3
```

Deploy dan periksa:

```
docker stack deploy -c kantin-stack.yml kantin
docker stack services kantin
docker stack ps kantin
```

Coba hapus stack dengan `docker stack rm kantin`, lalu buka aplikasi lagi setelah di-deploy ulang. Data tetap ada karena tersimpan di instance `db`.

Pertanyaan Refleksi

Jawab dalam laporan:

1. Mengapa port 2377, 7946, 4789, dan 3306 hanya dibuka untuk security group sendiri, sedangkan 8080 dibuka untuk semua?
2. Apa keuntungan menempatkan MariaDB di instance sendiri dibanding sebagai service swarm yang dipasang di manager? Apa kekurangannya?
3. Sebutkan 2 kelemahan bind mount pada cluster multi-node dan 2 alternatif penggantinya.
4. Mengapa `DB_HOST` memakai IP private instance `db`, bukan IP public? Apa yang harus diubah bila instance database diganti dengan instance baru?
5. Apa perbedaan `docker service create` dengan `docker stack deploy`? Kapan sebaiknya memakai masing-masing?
6. Jika node manager mati, apa yang terjadi pada cluster dan pada aplikasi? Bagaimana cara meningkatkan ketersediaannya?

Troubleshooting

Gejala	Penyebab umum	Solusi
`docker swarm join` timeout atau connection refused	Port 2377 belum dibuka antar node, atau memakai IP public	Pakai IP private manager; pastikan rule TCP 2377 bersumber `sg-swarm`
`could not choose an IP address to advertise`	Node punya lebih dari satu IP	Tambahkan `--advertise-addr <IP-PRIVATE>`
Task service `Rejected`, pesan `bind source path does not exist`	Folder bind mount belum ada di node tempat replika dijadwalkan	Buat folder dan file yang sama di semua node
Halaman hanya terbuka dari sebagian node, atau replika tidak merespons	Port TCP/UDP 7946 atau UDP 4789 belum dibuka	Periksa inbound rules `sg-swarm`

Gejala	Penyebab umum	Solusi
Browser tidak bisa membuka `IP:8080`	Rule 8080 belum ada, atau IP public berubah setelah lab di-restart	Tambahkan rule 8080; cek IP public terbaru di console
`permission denied` pada `docker.sock`	Sesi SSH belum dimuat ulang setelah `usermod`	Keluar dari SSH lalu masuk lagi
`This node is not a swarm manager`	Perintah manager dijalankan di worker	Jalankan di node manager
Uji koneksi database timeout atau `Connection refused`	Rule TCP 3306 belum dibuka, instance `db` tidak memakai `sg-swarm`, atau container `db` mati	Periksa rule 3306; di instance `db` jalankan `docker ps`, lalu `docker start db` bila perlu
appKantin gagal terhubung ke database	`DB_HOST` di `config/db.php` bukan IP private instance `db`, atau berbeda antar node	Samakan `db.php` di ketiga node dengan IP private yang benar

Hemat Credit dan Pembersihan

- Saat berhenti praktik, klik **End Lab** di Learner Lab. Instance akan di-stop dan IP public berubah saat lab dijalankan lagi. IP private tetap, dan cluster swarm biasanya pulih sendiri.
- Jika praktik sudah selesai seluruhnya dan data tidak diperlukan lagi, terminate keempat instance dan hapus security group `sg-swarm`.
- Jangan menyimpan password nyata di `config/db.php`. Password `rahasia` hanya untuk latihan.

Laporan dan Penilaian

Laporan dibuat dalam satu berkas dan dikumpulkan di GC. Isi laporan:

1. Identitas (nama, kelas, kelompok) dan tujuan praktik.
2. Tangkapan layar: console EC2 (4 instance dengan nama dan IAM role), `docker node ls`, `docker service ps web` (kolom NODE), browser menampilkan hostname yang berbeda, aplikasi kantin berjalan, dan hasil query database.
3. Tabel pengamatan Langkah 6 yang sudah terisi.
4. Jawaban Pertanyaan Refleksi.
5. Kendala yang dialami dan cara mengatasinya, serta kesimpulan.

Aspek yang dinilai	Bobot
Persiapan AWS: security group, 4 instance, LabInstanceProfile	15
Cluster swarm 1 manager + 2 worker berfungsi	20

Aspek yang dinilai	Bobot
Service web 3 replika dan bukti routing mesh	20
appKantin terhubung ke MariaDB di instance tersendiri	25
Jawaban pertanyaan refleksi	10
Kelengkapan dan kerapian laporan	10
Total	**100**